

Fire and Smoke Detection Report

Abstract

This paper presents a comparative analysis of various deep learning models for fire and smoke detection using semantic segmentation. Several models were evaluated, including UNet, YOLOv7, YOLOv5 with segmentation, DeepLabV3+, Mask R-CNN, Detectron2, and Segment Anything Model (SAM). The goal was to determine the most suitable model for deployment on an NVIDIA Jetson Nano 2GB. The results show that YOLOv11 outperformed other models in terms of accuracy, latency, and deployment feasibility.

Introduction

Fire detection is a critical application of computer vision, requiring real-time performance and high accuracy to minimize losses. Traditional fire detection methods, such as smoke sensors and thermal cameras, suffer from limitations including false alarms and high costs. Deep learning models offer a promising alternative by enabling real-time fire and smoke detection using standard RGB cameras. This study compares various deep learning architectures to identify the most efficient model for real-world deployment.

Related Work

Several deep learning models have been used for fire detection. UNet and DeepLabV3+ are common choices for semantic segmentation but may struggle with high real-time processing demands. YOLO-based models offer a trade-off between speed and accuracy. Recent models like SAM and Mask R-CNN provide advanced segmentation but are computationally intensive.

Pipeline

dataset collected from the FASDD dataset, filtered to 1032 images after augmentation,

augmentations performed- only rotations(90,180,270)

annotations made in labelme

dataset collected in the formats--(coco json, json, txt, xml) all polygons

1. Data Annotation

- **Tools Used:** LabelMe or similar annotation tools.

Command to download the tool-

```
pip3 install labelme
```

- **Process:**
- Annotated 250 images for both fire types and smoke.
- **Fire Classification:**
- **Class A:** Combustible solids (wood, paper, etc.).
- **Class B:** Flammable liquids (oil, gasoline, etc.).
- **Class C:** Electrical fires (due to wiring, electrical equipment).
- Smoke regions were labeled as distinct segments.
- **Strategy:**
- Annotating fuel sources explicitly was avoided to simplify labeling, relying on visual features to classify fire types.

2. Dataset Preparation

- **Image Augmentation:**
- Techniques: Rotation, flipping, and brightness adjustments.
- Purpose: To increase dataset diversity and improve model robustness.
- **Label Conversion:**
- Converted annotations to a model-compatible format (COCO format and txt labels).
- **Class Labeling:**
- Fire images were labeled as Class A, B, or C based on visible features and context.

3. Model Training

- **Models Tested:**
- UNet
- YOLOv11 (Lite Version)
- Detectron2
- MaskRCNN
- SAM(Segment anything model)
- Deeplabv3+
- YOLOV5 with segmentation
- YOLOV7 with segmentation
- **Task:** Semantic Segmentation.

- **Performance Summary:**
- **YOLOv11:** Achieved the best performance in terms of accuracy and latency.
- Other models failed to infer effectively or did not work within the constraints of the Jetson Nano environment.
- **Training Parameters:**
- Trained all models for 300 epochs or equivalent iterations.

4. Deployment

- **Target Device:** NVIDIA Jetson Nano.
- **Challenges:**
- Models other than YOLO could not be deployed due to deprecated support for JetPack 4.
- YOLOv11 (Lite Version) was successfully deployed using a Docker container.
- **Performance on Jetson Nano:**
- Decent results on both video and image inference.
- Some latency observed but within acceptable limits.

5. Models Evaluated

- UNet: A well-known segmentation model, but failed to provide satisfactory results.
- Detectron2: Did not converge properly on the dataset.
- Auto-labeling with UNet (Smoke Detection): Used to generate segmentation masks for smoke; however, it did not achieve sufficient accuracy.
- SAM: High computational requirements made it infeasible for Jetson Nano.
- YOLOv7 & YOLOv5 (Segmentation versions): Accuracy was inadequate for reliable deployment.
- DeepLabV3+: Produced reasonable segmentation but had high latency.
- Mask R-CNN: Achieved moderate accuracy but was too slow for real-time inference.
- YOLOv11 (Lite Version): Achieved the best accuracy and inference speed, making it ideal for deployment.

5.1 Training and Evaluation

- Models trained for 300 epochs or equivalent iterations.
- Performance metrics: Accuracy, Intersection-over-Union (IoU), and inference latency.

Steps for Deployment On Jetson nano

After flashing the sd card with the image, we have to set it up in headless mode only, directly booting with display won't work. after the initial setup we can boot it with display and keyboard.

##link for ubuntu image--

<https://developer.nvidia.com/jetson-nano-2gb-sd-card-image>

##link for ultralytics docker repo--

<https://docs.ultralytics.com/guides/nvidia-jetson/>

YOLOv11 Setup and Run Instructions for NVIDIA Jetson Nano with JetPack 4

1. System Update and Swapfile Configuration

1. Update and upgrade the system:

```
sudo apt-get update
```

```
sudo apt-get upgrade
```

2. Install Nano:

```
$ sudo apt-get install nano
```

3. Install `dphys-swapfile` to manage swap:

```
$ sudo apt-get install dphys-swapfile
```

4. Enlarge the swap boundary:

- Edit the `/sbin/dphys-swapfile` file:

```
$ sudo nano /sbin/dphys-swapfile
```

- Set `CONF\_MAXSWAP=10240`

5. Configure the required memory size:

- Edit the `/etc/dphys-swapfile` file:

```
$ sudo nano /etc/dphys-swapfile
```

- Set `CONF\_SWAPSIZE=10240`

6. Reboot the system:

```
$ sudo reboot
```

Installing YOLOv11 with docker

1. Pull the YOLOv11 Docker image:

```
$ t=ultralytics/ultralytics:latest-jetson-jetpack4
sudo docker pull $t && sudo docker run -it --ipc=host --runtime=nvidia $t
$ exit
```

2. Start the Docker container with necessary devices and permissions:

```
$ sudo docker run -it --runtime nvidia \
--device /dev/video0 \
--device /dev/nvhost-ctrl \
--device /dev/nvhost-ctrl-gpu \
--device /dev/nvhost-prof-gpu \
--device /dev/nvmap \
--device /dev/nvhost-gpu \
--device /dev/nvhost-vic \
--device /dev/nvhost-nvdec \
--device /dev/nvhost-msenc \
--device /dev/nvhost-isp \
--device /dev/nvhost-vi \
--volume /tmp/argus_socket:/tmp/argus_socket \
-v ~/myenv/FYP:/workspace \
-e DISPLAY=$DISPLAY \
-e QT_X11_NO_MITSHM=1 \
-v /tmp/.X11-unix:/tmp/.X11-unix \
--privileged \
ultralytics/ultralytics:latest-jetson-jetpack4
```

##the above step will directly get into the docker the below step is for starting it again.(we don't need to use the above command each and every time, the above command is to setup the docker with permissions for camera access).

3. Running YOLOv11 in the Docker Container

1. Check the Docker container ID:

```
$ sudo docker ps -a
```

2. Start the container:

\$ sudo docker start 04ceab8bc12f #this is the id of the existing docker change it for the newly installed one

3. Access the container's shell:

\$ sudo docker exec -it 04ceab8bc12f /bin/bash

4. Run YOLOv11 for segmentation:

\$ yolo task=segment mode=predict model="ultralytics/send/best.pt" source=0 save=True save_txt=True project="ultralytics/runs" name="prediction_results"

YOLOv7 Setup and Run Instructions for NVIDIA Jetson Nano with JetPack 4

1. Installing YOLOv7 Inside the Ultralytics Docker Container

For installing yolov7 on jetson, we need to install ultralytics first and then install yolov7 in that docker container. Individually installing the yolo7 won't work.

1. Update and upgrade the system:

sudo apt-get update

sudo apt-get upgrade

2. Install Nano:

\$ sudo apt-get install nano

3. Install `dphys-swapfile` to manage swap:

\$ sudo apt-get install dphys-swapfile

4. Enlarge the swap boundary:

- Edit the `/sbin/dphys-swapfile` file:

\$ sudo nano /sbin/dphys-swapfile

- Set `CONF\_MAXSWAP=10240`

5. Configure the required memory size:

- Edit the `/etc/dphys-swapfile` file:

```
$ sudo nano /etc/dphys-swapfile
```

```
- Set `CONF_SWAPSIZE=10240`
```

6. Reboot the system:

```
$ sudo reboot
```

2. Installing YOLOv11 with Docker

1. Pull the YOLOv11 Docker image:

```
$ t=ultralytics/ultralytics:latest-jetson-jetpack4
```

```
sudo docker pull $t && sudo docker run -it --ipc=host --runtime=nvidia $t
```

```
$ exit
```

2. Start the Docker container with necessary devices and permissions:

```
$ sudo docker run -it --runtime nvidia \  
--device /dev/video0 \  
--device /dev/nvhost-ctrl \  
--device /dev/nvhost-ctrl-gpu \  
--device /dev/nvhost-prof-gpu \  
--device /dev/nvmap \  
--device /dev/nvhost-gpu \  
--device /dev/nvhost-vic \  
--device /dev/nvhost-nvdec \  
--device /dev/nvhost-msenc \  
--device /dev/nvhost-isp \  
--device /dev/nvhost-vi \  
--volume /tmp/argus_socket:/tmp/argus_socket \  
-v ~/myenv/FYP:/workspace \  
-e DISPLAY=$DISPLAY \  
-e QT_X11_NO_MITSHM=1 \  
-v /tmp/.X11-unix:/tmp/.X11-unix \  
--privileged \  
ultralytics/ultralytics:latest-jetson-jetpack4
```

1. Access the Docker container shell for YOLOv11:

```
sudo docker exec -it <container_id_or_name> /bin/bash
```

2. Clone the YOLOv7 repository:

```
cd /ultralytics
```

```
git clone https://github.com/WongKinYiu/yolov7.git
```

3. Download the pre-trained weights:

```
cd yolov7
```

```
wget https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7.pt
```

2. Running YOLOv7 in the Docker Container

1. Check the Docker container ID:

```
sudo docker ps -a
```

2. Start the container:

```
sudo docker start <container_id_or_name>
```

3. Access the container's shell:

```
sudo docker exec -it <container_id_or_name> /bin/bash
```

4. Run YOLOv7 for detection:

```
python3 detect.py --weights traced_model.pt --source 0
```

5. Alternative detection with `yolov7.pt`:

```
python3 detect.py --weights yolov7.pt --source 0
```

Push files from local to docker container:

from system to docker::

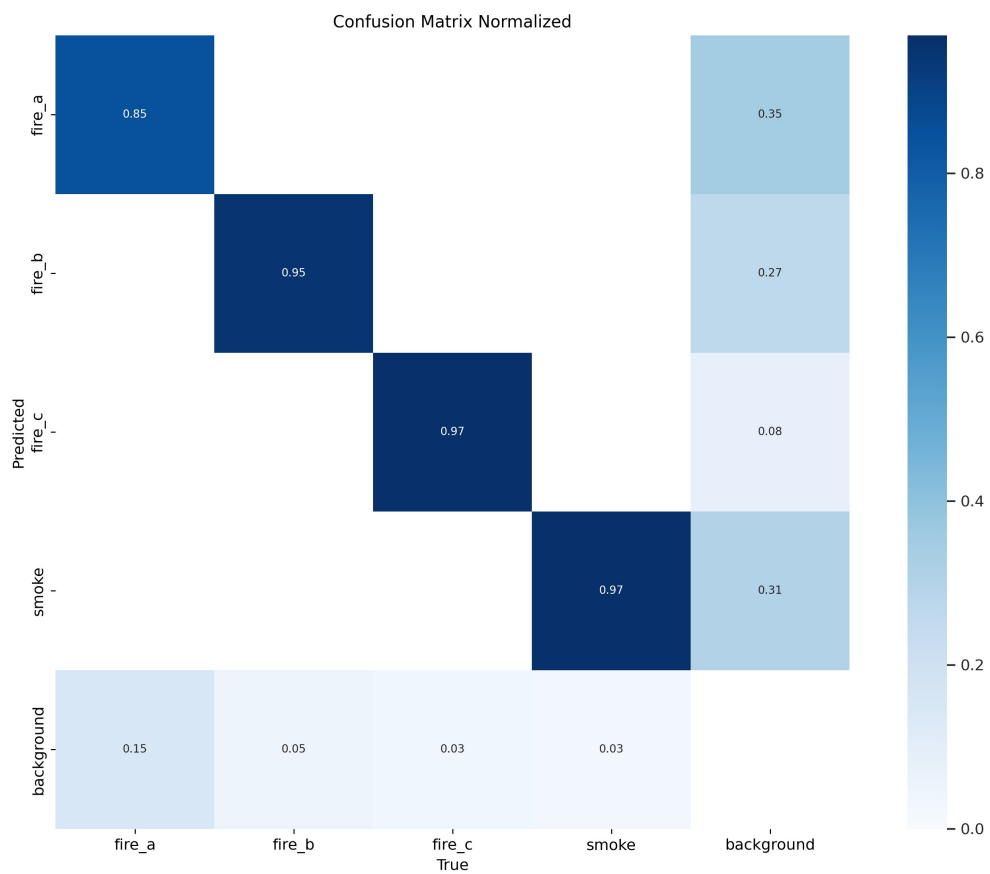
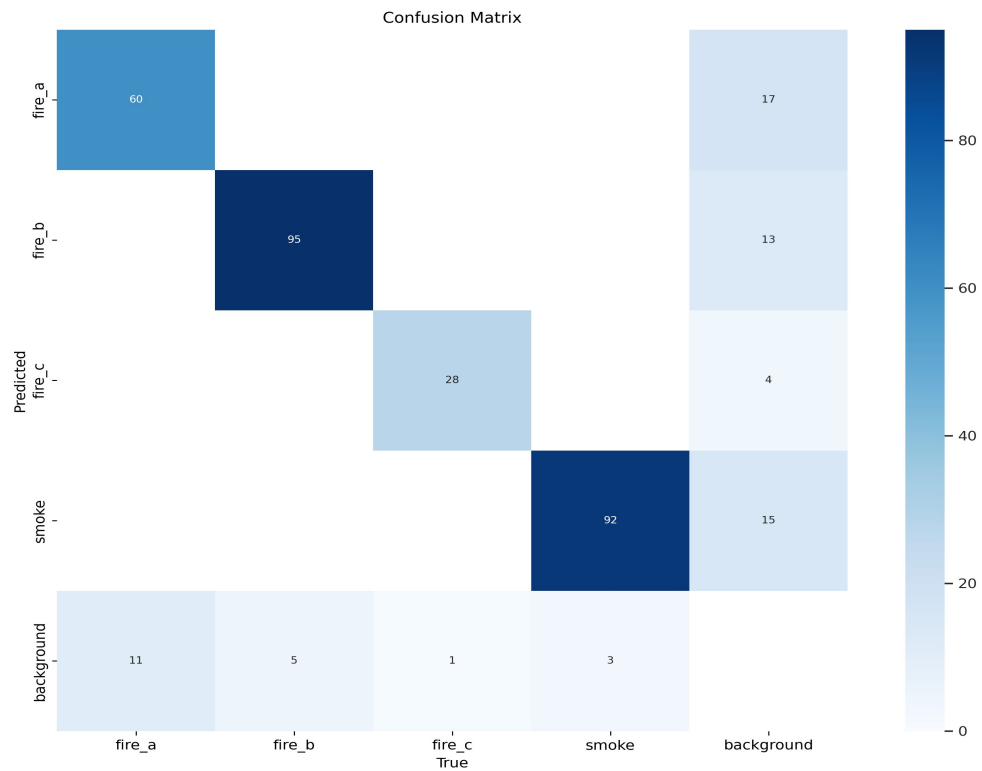
```
$sudo docker cp /home/ubuntu/v7_onjetson <container_id_or_name>:/ultralytics/ultralytics/yolov7
```

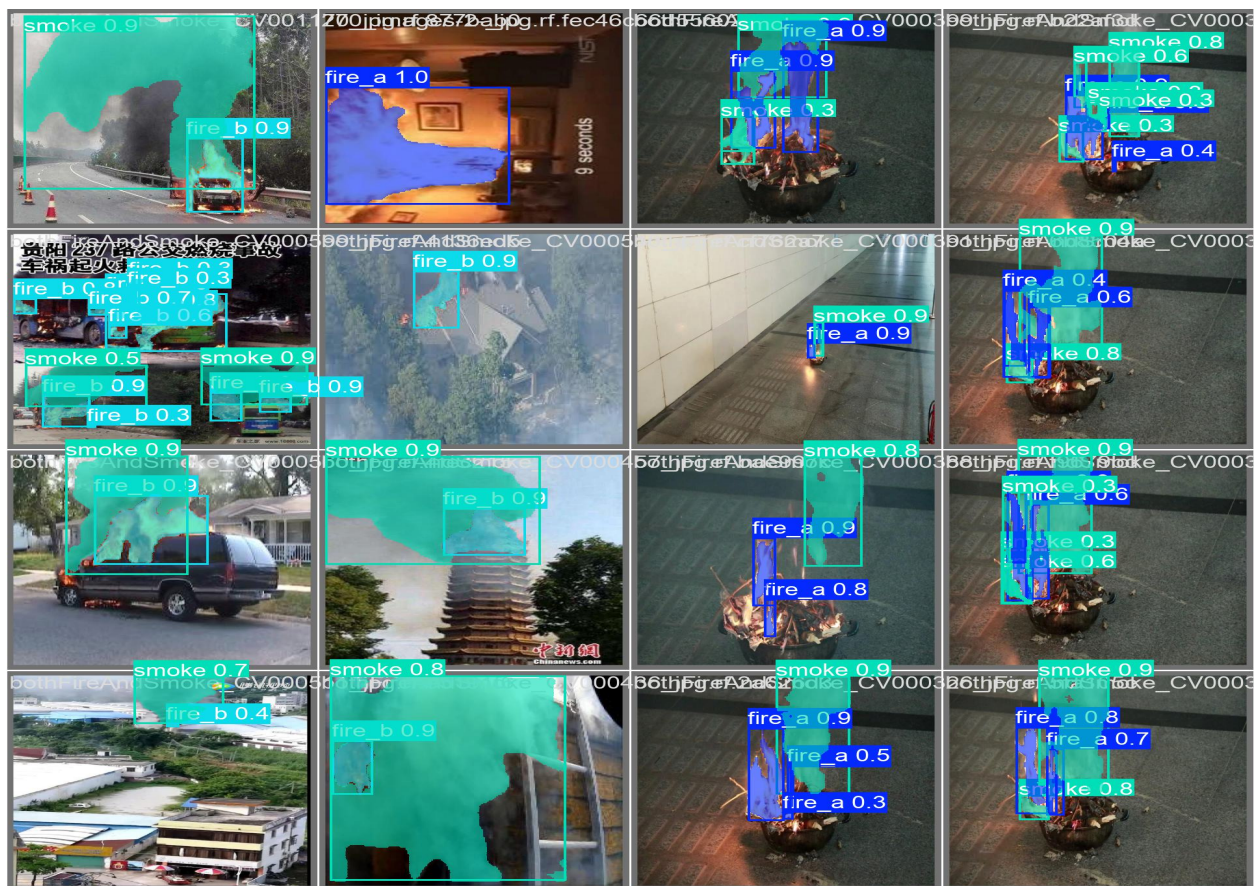
from docker to system::


```
$sudo docker cp <container_id>:/path/to/folder/on/container /path/to/destination/on/host
```

Evaluation Metrics

- **Accuracy:** YOLOv11 achieved the highest accuracy for both fire and smoke detection.
- **Latency:** Slight delays in real-time video processing were noted but acceptable for the use case.
- **Deployment Success:** YOLOv11 was the only model to run effectively on the Jetson Nano with JetPack 4 constraints.





Results

Video Inference done on a demo video.





Results and Comparison

Model	Accuracy	IoU	Latency	Deployment Feasibility
UNet	Low	Low	High	No
Detectron2	Failed	N/A	N/A	No
SAM	High	High	Very High	No
YOLOv5 (Seg)	Medium	Medium	Medium	No
YOLOv7	Medium	Medium	High	No
DeepLabV3+	Medium	High	High	No
Mask R-CNN	Medium	Medium	Very High	No
YOLOv11	High	High	Low	Yes

Conclusion

This project successfully implemented a fire and smoke detection system using semantic segmentation. YOLOv11 proved to be the most viable model for both performance and deployment. Future improvements can focus on optimizing latency and expanding the dataset for better generalization.

Future Work

- Explore lightweight model optimizations for lower latency.
- Expand the dataset for better generalization.
- Implement multi-sensor fusion techniques for enhanced accuracy.